Integración de APIs y flujo de interacción de Wally Reservas

Documento técnico — 8 páginas Sistema: Wally Reservas **Tecnología principal:** Laravel 12, PHP 8.2+, MySQL, Blade y JavaScript **Fecha de revisión:** 11 de septiembre de 2026

Página 1 de 8 — Propósito, alcance y resumen ejecutivo

Este documento explica qué APIs usa Wally Reservas, cómo se configuran, en qué archivos se conectan y cuál es el recorrido de los datos desde que una persona interactúa con la interfaz hasta que el sistema consulta o modifica la base de datos. La revisión se basa en el código real del proyecto y no expone valores secretos del archivo `.env`.

En el sistema existen cuatro integraciones externas que deben diferenciarse. La primera es **Google OAuth para autenticación**, usada para iniciar sesión o vincular una cuenta Google. La segunda es **Google Calendar API**, usada por la administración para reflejar reservas confirmadas, canceladas o completadas en un calendario empresarial. Aunque ambas pertenecen a Google, tienen credenciales, permisos, rutas de retorno y propósitos distintos. La tercera integración es el **proveedor NLP del chatbot**: actualmente la configuración selecciona Gemini, aunque el código también admite OpenAI. Su función se limita a detectar la intención y extraer entidades como cancha, fecha y hora. La cuarta es la **Web Speech API del navegador**, que convierte voz a texto y lee la respuesta de WallyBot.

Integración	Finalidad	Punto de conexión principal	Estado observado
Google OAuth / Socialite	Autenticar o vincular usuarios	GoogleAuthController.php	Configurada mediante variables de entorno
Google Calendar API	Crear, actualizar, verificar y eliminar eventos	GoogleCalendarService.php	Habilitada por configuración y administrada por OAuth
Gemini u OpenAI	Analizar lenguaje natural del chatbot	ProveedorExterno.php	Gemini seleccionado; existe respaldo local
Web Speech API	Entrada y salida por voz	resources/js/chatbot.js	Depende del navegador, micrófono y contexto seguro

También existen configuraciones estándar para Postmark, Resend, Amazon SES y Slack en `config/services.php`, pero no se encontraron llamadas de la aplicación que las utilicen en estos flujos. El correo está configurado con el transportador `log`, por lo que no debe presentarse como una API externa activa. `GOOGLE_API_KEY` existe en el entorno, pero el código revisado no la consume: Calendar usa OAuth y Gemini usa `GEMINI_API_KEY`.

La idea central de la arquitectura es: **la base de datos local es la fuente de verdad**. Ni Gemini ni Google Calendar deciden si una cancha está libre, cuánto cuesta o quién puede reservar. Laravel consulta MySQL, aplica permisos y reglas de negocio, guarda la operación y recién después sincroniza con servicios externos. Si Gemini falla, continúa el motor local. Si Calendar falla, la reserva local no se pierde y el error queda preparado para reintento.

Página 2 de 8 — Arquitectura general y archivos que se comunican

Wally Reservas sigue una arquitectura por capas. La persona interactúa con una vista Blade; JavaScript puede realizar una petición; una ruta de Laravel dirige la solicitud a un controlador; el controlador valida y delega a un servicio; el servicio consulta modelos Eloquent y, cuando corresponde, llama a una API externa. La respuesta recorre el camino inverso hasta mostrarse en el navegador.

```
Usuario
  → Vista Blade / JavaScript
  → Ruta web + middleware
  → Controlador
  → Servicio de aplicación
  → Modelos Eloquent ⇔ MySQL
  → API externa (solo cuando corresponde)
  → JSON o redirección → interfaz del usuario
```

Los archivos de arranque y configuración son `.env`, `config/services.php`, `config/chatbot.php`, `config/database.php`, `config/queue.php`, `config/session.php`, `bootstrap/app.php` y `app/Providers/AppServiceProvider.php`. `.env` contiene los valores locales; los archivos de `config/` asignan esos valores a nombres que usa Laravel. `AppServiceProvider` enlaza la interfaz `GoogleCalendarGateway` con `GoogleCalendarService` y define el límite de peticiones del chatbot. `bootstrap/app.php` registra los alias de middleware `admin` y `recepcionista`.

Las rutas se reparten por responsabilidad. `routes/web.php` carga autenticación y los módulos. `routes/publico.php` expone `POST /chatbot/mensaje`. `routes/cliente.php` contiene reservas del cliente y vinculación de Google. `routes/admin.php` protege la administración y la integración Calendar con `auth` y `admin`. `routes/recepcionista.php` protege operaciones de recepción.

Flujo	Presentación	Controlador / servicio	Persistencia
Chat escrito o por voz	widget.blade.php, chatbot.js, layouts	ChatbotController, ChatbotService, clases de Services/Chatbot/	conversaciones, mensajes_chatbot, catálogos y reservas
Reserva	reservas/crear.blade.php, chatbot	ReservaController, ReservaConversacionalService, ReservaService	users, canchas, horarios, balones, reservas
Login Google	vistas de autenticación y perfil	GoogleAuthController, Laravel Socialite	users, social_accounts, sessions
Calendar	admin/integraciones/google-calendar.blade.php	GoogleCalendarController, Jobs, GoogleCalendarService	google_calendar_connections, google_calendar_events, jobs, failed_jobs

La inyección de dependencias evita que los controladores conozcan detalles HTTP. Por ejemplo, `GoogleCalendarController` recibe `GoogleCalendarGateway`; el contenedor entrega `GoogleCalendarService` según `AppServiceProvider`. Esto permite simular la API en pruebas y reduce el acoplamiento.

La aplicación usa sesiones en base de datos. En el chatbot, el identificador de conversación se guarda en sesión; en OAuth se guardan temporalmente la intención de login/vinculación y el valor de seguridad `state`. Para usuarios autenticados, la conversación se asocia además con `user_id`. Así se mantiene contexto sin mezclar conversaciones privadas entre personas.

Página 3 de 8 — API de Google OAuth para inicio de sesión

El acceso con Google se implementa con `laravel/socialite`, declarado en `composer.json`. Su configuración nace en `.env` mediante `GOOGLE_AUTH_CLIENT_ID`, `GOOGLE_AUTH_CLIENT_SECRET` y `GOOGLE_AUTH_REDIRECT_URI`; `config/services.php` las reúne bajo `services.google`. Los secretos permanecen en el servidor y no se envían a las vistas ni al JavaScript.

El flujo comienza en `routes/web.php` con `GET /auth/google` y `GET /auth/google/callback`. Para vincular una cuenta existente, `routes/cliente.php` añade `GET /perfil/google/vincular` y `DELETE /perfil/google`. La vinculación requiere `auth`; el login inicial es para visitantes.

```
1. Usuario pulsa “Continuar con Google”.
2. GoogleAuthController::redirect() registra intención “login”.
3. Socialite redirige a Google con scopes openid, profile y email.
4. Google autentica y devuelve un código al callback.
5. Socialite intercambia el código y obtiene el perfil verificado.
6. Laravel busca social_accounts o crea users + social_accounts.
7. Auth::login() inicia sesión, regenera la sesión y redirige según rol.
```

`app/Http/Controllers/Auth/GoogleAuthController.php` concentra las decisiones. Antes de redirigir comprueba que las tres variables requeridas estén definidas. Guarda `google_oauth_intent` en sesión para distinguir `login` de `link`. En el retorno captura errores, exige identificador, correo y marca de correo verificado, y luego llama a `loginOrRegister()` o `linkAuthenticatedUser()`.

Para un login ya vinculado, consulta `SocialAccount` por `provider = google` y `provider_user_id`. Si el usuario está inactivo, rechaza el acceso. Si no existe vínculo, busca primero el correo en `users`: cuando ya existe, no lo fusiona automáticamente, sino que pide iniciar con contraseña y vincular desde el perfil. Esto evita apropiaciones accidentales de cuentas. Solo si el correo no existe crea, dentro de `DB::transaction`, un usuario cliente activo con contraseña nula y su vínculo social.

Las tablas se definen en `2026_09_04_000001_make_user_password_nullable.php` y `2026_09_04_000002_create_social_accounts_table.php`. `users.email` es único. `social_accounts` impone unicidad sobre `(provider, provider_user_id)` y sobre `(user_id, provider)`: una cuenta Google no puede pertenecer a dos usuarios y un usuario no puede registrar dos vínculos del mismo proveedor. Al eliminar el usuario, el vínculo se elimina en cascada.

La desvinculación también tiene una restricción: `unlink()` exige que el usuario tenga contraseña local; de lo contrario quedaría sin forma de entrar. El sistema no guarda tokens OAuth de login en `social_accounts`, solo el identificador externo, correo y URL del avatar. Finalmente, `finishLogin()` regenera la sesión contra fijación de sesión y redirige a cliente, recepción o administración según el rol guardado localmente; Google no asigna privilegios.

Archivos clave: `config/services.php`, `composer.json`, `routes/web.php`, `routes/cliente.php`, `GoogleAuthController.php`, `User.php`, `SocialAccount.php`, las dos migraciones indicadas, `resources/views/auth/login.blade.php` y `resources/views/profile/edit.blade.php`.

Página 4 de 8 — Google Calendar API y sincronización por colas

Calendar utiliza un OAuth distinto del login. Las variables `GOOGLE_CALENDAR_CLIENT_ID`, `GOOGLE_CALENDAR_CLIENT_SECRET`, `GOOGLE_CALENDAR_REDIRECT_URI`, `GOOGLE_CALENDAR_ID` y `GOOGLE_CALENDAR_ENABLED` se leen en `config/services.php`. El cliente y secreto pueden heredar los de autenticación si los específicos faltan, pero la URI de retorno de Calendar debe configurarse. Los permisos solicitados son `calendar.events` y lectura de la lista de calendarios.

La pantalla `resources/views/admin/integraciones/google-calendar.blade.php` se abre mediante rutas de `routes/admin.php`, todas protegidas por `auth` y `admin`. `GoogleCalendarController::connect()` crea un `state` aleatorio de 64 caracteres y lo guarda en sesión. `GoogleCalendarService::authorizationUrl()` construye la URL de Google con acceso offline. En el callback, el controlador compara el `state` recibido con `hash_equals`, intercambia el código por tokens y persiste la conexión. `GoogleCalendarConnection.php` cifra `access_token` y `refresh_token` con casts `encrypted` antes de almacenarlos.

La sincronización no bloquea la pantalla del usuario. `ReservaService::programarSincronizacionGoogle()` marca un vínculo como `pending` y despacha `SyncReservationToGoogleCalendar` después de confirmar la transacción. Con `QUEUE_CONNECTION=database`, el trabajo queda en `jobs` hasta que `php artisan queue:work` lo procese. El Job intenta hasta cuatro veces, espera 60, 300 y 900 segundos entre reintentos y usa `WithoutOverlapping` para evitar dos sincronizaciones simultáneas de la misma reserva.

```
Reserva cambia de estado
  → ReservaService
  → google_calendar_events = pending
  → jobs
  → SyncReservationToGoogleCalendar
  → GoogleCalendarGateway
  → GoogleCalendarService
  → Calendar REST API
  → vínculo local = synced / cancelled / error
```

Las reservas `pendiente` no generan evento. Al pasar a `confirmada`, el servicio crea o actualiza un evento privado con cancha, número de reserva, estado, cliente, balón, observaciones, ubicación, inicio y fin en `America/La_Paz`. Una cancelación o finalización actualiza el mismo evento y añade el estado visible; no lo borra. Solo la eliminación física administrativa dispara `DeleteGoogleCalendarEvent`.

`GoogleCalendarService.php` llama a tres endpoints: autorización, token y REST `/calendar/v3`. Usa Bearer token, renueva el acceso con el refresh token cuando está por vencer, verifica el calendario, crea con POST, actualiza con PUT y elimina con DELETE. Trata 404/410 de borrado como resultado aceptable. Ante 404 de actualización vuelve a crear; ante 409 actualiza el ID existente.

La idempotencia se apoya en un ID determinista calculado con `APP_KEY` y `reserva_id`, el vínculo único por reserva y el índice único `(calendar_id, google_event_id)` agregado por `2026_09_11_000001_harden_google_calendar_event_identity.php`. `payload_hash`, `etag`, URL externa, fecha de sincronización y último error permiten auditoría. Un error externo cambia el vínculo a `error` y hace fallar/reintentar el Job, pero no revierte ni modifica la reserva local.

Archivos clave: `GoogleCalendarGateway.php`, `GoogleCalendarService.php`, `GoogleCalendarController.php`, `SyncReservationToGoogleCalendar.php`, `DeleteGoogleCalendarEvent.php`, `ReservaService.php`, ambos modelos Calendar, migraciones `2026_09_04_000003`, `000004` y `2026_09_11_000001`, `config/queue.php` y la vista administrativa.

Página 5 de 8 — API de Gemini/OpenAI y motor seguro de WallyBot

El chatbot combina inteligencia externa opcional con un motor local determinista. `.env` selecciona `CHATBOT_PROVIDER`; la revisión encontró `gemini`, con modelo configurado y `GEMINI_API_KEY` definida. `config/chatbot.php` centraliza nombre del bot, proveedor, clave, modelo, contacto y límites. El mismo código admite `openai` y conserva `CHATBOT_API_KEY` como compatibilidad, pero solo un proveedor se usa por petición.

`app/Services/Chatbot/ProveedorExterno.php` realiza la llamada. Para Gemini envía un POST a `generativelanguage.googleapis.com/v1beta/models/{modelo}:generateContent` con `x-goog-api-key`. Para OpenAI envía Bearer token a `/v1/chat/completions`. Ambas variantes tienen timeout de 15 segundos, temperatura cero y exigen una salida JSON estructurada. No hay reintento dentro de la petición del usuario.

El proveedor recibe únicamente el mensaje actual y una instrucción que le prohíbe afirmar precios, disponibilidad, clientes o reservas. Debe devolver: `intencion`, `reserva_id`, `cancha_referencia`, `fecha`, `hora_inicio`, `hora_fin`, `duracion_minutos` y `balon_referencia`. `AnalisisNlp.php` valida el catálogo de intenciones, longitudes, enteros, fecha `Y-m-d`, horas y duración de 15 a 720 minutos. Si hay HTTP no exitoso, excepción, JSON inválido o configuración incompleta, devuelve `null`, registra un aviso sin incluir la clave y continúa con el análisis local.

El motor local se compone de `DetectorIntencion.php` y `ExtractorEntidades.php`. El primero normaliza mayúsculas, tildes y puntuación, y reconoce saludo, despedida, ayuda, contacto, horarios, precios, disponibilidad, creación, consulta, modificación y cancelación. El segundo interpreta fechas relativas, días, horas y referencias; para las canchas consulta nombres reales en la base de datos.

`ChatbotService.php` orquesta ambos resultados. Calcula siempre un análisis local; si no existe un borrador de reserva, intenta el proveedor externo y le da prioridad a su intención. Después vuelve a resolver cada referencia contra modelos Eloquent. La IA puede sugerir “cancha 1”, pero solo `ExtractorEntidades` y `Cancha::disponibles()` determinan si esa cancha existe y está habilitada. Precios, horarios, balones y estados se obtienen desde MySQL y la respuesta final se construye localmente.

```
Mensaje → análisis local
       ↘ proveedor externo opcional → DTO AnalisisNlp validado
Resultados → resolver entidades en MySQL → regla de negocio → respuesta local
```

Cuando hay un borrador activo, confirmaciones, negativas o elección de balón omiten Gemini. Esto evita llamadas innecesarias y asegura que “sí, confirmar” se interprete dentro del flujo controlado. Modificación y cancelación conversacional se reconocen, pero actualmente no ejecutan cambios; el bot dirige al usuario a “Mis reservas”. Las solicitudes de varias franjas también se detectan, pero se rechazan explícitamente sin escribir datos.

La seguridad conceptual es importante: una respuesta del modelo no se ejecuta como SQL, no elige IDs directamente, no conoce secretos y no puede confirmar una reserva. La IA mejora la comprensión del lenguaje; Laravel conserva autoridad total sobre datos y operaciones.

Página 6 de 8 — Interacción con el usuario: chat, voz y reserva conversacional

El widget se define en `resources/views/chatbot/widget.blade.php` y se incluye desde `resources/views/layouts/public.blade.php` y `resources/views/layouts/app.blade.php`. `resources/js/app.js` importa `chatbot.js`; Vite lo compila según `vite.config.js`. El botón de voz solo se renderiza para un usuario autenticado, activo y con rol cliente.

Al escribir, `resources/js/chatbot.js` intercepta el formulario y ejecuta `fetch()` hacia `POST /chatbot/mensaje`. Envía JSON `{mensaje: texto}`, cabecera `Accept: application/json` y token CSRF. `routes/publico.php` aplica `throttle:chatbot`. `AppServiceProvider.php` permite por defecto 10 mensajes por minuto a visitantes, identificados mediante hash de IP, y 30 a autenticados, identificados por usuario. `ChatbotController::mensaje()` exige texto, máximo 500 caracteres.

El controlador recupera `conversacion_id` desde sesión. Si la conversación pertenecía a otro usuario o el usuario cerró sesión, no la reutiliza. Si el visitante inicia sesión, puede asociar su conversación pública al usuario. Si no hay una válida, crea `conversaciones`; luego `ChatbotService` guarda el mensaje del usuario y la respuesta en `mensajes_chatbot`, incluyendo la intención. El administrador puede revisar el historial mediante `ConversacionController` y las vistas `admin/chatbot/`.

Para reservar, `ReservaConversacionalService.php` mantiene en `conversaciones.contexto` un borrador con cancha, fecha, horario, balón, estado y vencimiento. El bot solicita los datos faltantes en turnos sucesivos, muestra horarios y balones libres, calcula el total real y genera un resumen. El borrador vence en 10 minutos. Una reserva solo se escribe cuando el cliente activo responde de forma explícita “sí, confirmar”. Antes de guardar se vuelven a consultar cancha, fecha, horario y balón; si la disponibilidad cambió, se rechaza y se ofrecen alternativas.

La confirmación llama a `ReservaService::crearReserva()`, el mismo servicio usado por el formulario web. La nueva reserva queda `pendiente`, por lo que todavía no crea evento Calendar. Un administrador o recepcionista puede confirmarla posteriormente, momento en que se programa la sincronización externa.

La llamada de voz usa `SpeechRecognition` o `webkitSpeechRecognition` y `speechSynthesis` del navegador. El navegador solicita permiso del micrófono, escucha en `es-BO`, produce una transcripción y la envía al mismo endpoint del chat. Cuando Laravel devuelve texto, `SpeechSynthesisUtterance` lo lee y el ciclo vuelve a escuchar. Requiere Chrome/Edge compatible y HTTPS en producción o `localhost` en desarrollo. Si el permiso es rechazado, falta micrófono o hay error de red, finaliza la llamada y conserva el chat escrito.

Laravel no recibe archivos de audio ni guarda grabaciones: recibe y almacena la transcripción como un mensaje normal. Sin embargo, el servicio de reconocimiento del navegador puede procesar el audio según las políticas del navegador/proveedor, por lo que debe informarse al usuario. Al ocultar o abandonar la página se detienen reconocimiento y síntesis.

Página 7 de 8 — Base de datos, relaciones, restricciones y seguridad

La conexión activa usa MySQL mediante `config/database.php`; sesiones, caché y colas también usan tablas. Los modelos Eloquent traducen filas a objetos y definen relaciones. Las migraciones de `database/migrations/` documentan restricciones que siguen vigentes aunque la entrada provenga de formulario, chatbot o API externa.

Tabla	Función y relaciones	Restricciones principales
users	Personas y roles	correo único; rol cliente/recepcionista/admin; estado activo; contraseña puede ser nula para Google
social_accounts	Vínculo de login externo	FK a users con cascada; Google externo único; un proveedor por usuario
canchas / horarios	Catálogo y franjas	estados de cancha; FK horario con cascada; inicio único por cancha/día
balones	Catálogo compartido	tipo, tarifa y datos físicos; puede quedar nulo en reserva si se elimina
reservas	Operación principal	FKs; estados; índice anti-duplicidad por cancha/fecha/franja activa
conversaciones / mensajes_chatbot	Contexto e historial	usuario opcional; contexto JSON; mensajes en cascada con la conversación
google_calendar_connections	Credenciales empresariales	FK al administrador; tokens cifrados por el modelo
google_calendar_events	Relación local–externa	una fila por reserva; identidad Calendar única; reserva anulable al eliminarse
sessions, jobs, failed_jobs	Infraestructura Laravel	sesiones persistentes y ejecución/reintento asíncrono

Las validaciones comienzan en HTTP. `StoreReservaRequest.php` exige IDs existentes, fecha válida no pasada, balón opcional y observaciones de máximo 500 caracteres. `StoreRecepcionistaReservaRequest.php` además limita el cliente elegido a rol cliente y estado activo. `CambiarEstadoReservaRequest.php` acepta solo estados definidos por `Reserva::ESTADOS`. Los formularios de horario usan `SinSuperposicionHorario.php`: dos intervalos se superponen si el inicio de uno es anterior al fin del otro y viceversa; extremos contiguos sí se permiten.

La segunda defensa está en `ReservaService.php`. Dentro de una transacción, bloquea la cancha, horario y balón con `lockForUpdate`, exige cancha disponible, horario habilitado y correspondiente al día de la fecha, comprueba ocupación de cancha y balón y calcula el monto en el servidor. La columna generada `activa` y el índice único `reservas_sin_duplicados` impiden que dos reservas no canceladas ocupen la misma cancha, fecha y horario. Si dos solicitudes compiten, la excepción SQL se transforma en conflicto 409. Una cancelada libera la franja.

La autorización usa rutas, middleware y policies. `auth` exige sesión; `admin` llama `EnsureUserIsAdmin`; `recepcionista` permite recepción o administración. `ReservaPolicy.php` deja al cliente ver/cancelar solo lo suyo, reserva cambios de estado a personal operativo y eliminación a administración. Además, el chatbot solo confirma para clientes activos.

Otros controles: CSRF en formularios y `fetch`, regeneración de sesión tras OAuth, `state` en OAuth de Calendar, límite de mensajes, credenciales solo en `.env`, tokens Calendar cifrados, eventos privados y registros de error sin claves. En producción deben usarse `APP_DEBUG=false`, HTTPS, permisos mínimos del archivo `.env`, worker de colas supervisado y rotación de credenciales cuando hayan sido expuestas.

Página 8 de 8 — Flujos completos, operación y guía para exponer el sistema

Ejemplo 1: una persona reserva conversando. Abre el widget, escribe o dicta “quiero reservar la cancha 1 mañana a las 7”. `chatbot.js` envía la transcripción con CSRF; la ruta aplica rate limit; `ChatbotController` valida y obtiene la conversación. `ChatbotService` ejecuta análisis local y, si no hay borrador, puede consultar Gemini. La salida estructurada se valida, pero cancha, fecha y hora se resuelven localmente. `ReservaConversacionalService` consulta horarios y balones, completa el borrador y exige confirmación. Al recibir “sí, confirmar”, comprueba sesión, rol, vigencia y disponibilidad; `ReservaService` abre una transacción, bloquea recursos y crea `reservas` en estado pendiente. La respuesta JSON vuelve al navegador y se muestra o se lee por voz.

Ejemplo 2: la reserva llega a Google Calendar. Administración o recepción cambia la reserva a confirmada. El controlador autoriza la acción y llama a `ReservaService`, que actualiza MySQL, crea/actualiza `google_calendar_events` como pending y despacha el Job después del commit. El worker obtiene o renueva el token cifrado, convierte fecha y horario a RFC 3339 en `America/La_Paz` y envía el evento privado. Si Google responde correctamente, guarda ID, enlace, etag, hash y fecha. Si falla, conserva la reserva y marca error para reintento. Cancelar o completar actualiza el mismo evento; borrar físicamente la reserva programa su eliminación externa.

Ejemplo 3: inicio de sesión con Google. La vista dirige a `/auth/google`; Socialite solicita identidad básica; Google retorna al callback; el controlador exige correo verificado, consulta `social_accounts`, crea el usuario solo cuando no hay conflicto y regenera la sesión. El rol y el estado siempre provienen de MySQL.

Mapa final de archivos para mencionar en la exposición

- Configuración: `.env`, `config/services.php`, `config/chatbot.php`, `config/database.php`, `config/queue.php`, `config/session.php`.
- Arranque y rutas: `bootstrap/app.php`, `app/Providers/AppServiceProvider.php`, `routes/web.php`, `routes/publico.php`, `routes/cliente.php`, `routes/recepcionista.php`, `routes/admin.php`.
- Interfaces: `resources/views/chatbot/widget.blade.php`, `resources/js/chatbot.js`, `resources/js/app.js`, `resources/views/admin/integraciones/google-calendar.blade.php` y layouts.
- Controladores: `ChatbotController.php`, `GoogleAuthController.php`, `Admin/GoogleCalendarController.php`, `ReservaController.php` y controladores de reserva de administración/recepción.
- Servicios: `ChatbotService.php`, `Services/Chatbot/ProveedorExterno.php`, `AnalisisNlp.php`, `DetectorIntencion.php`, `ExtractorEntidades.php`, `ReservaConversacionalService.php`, `ReservaService.php`, `GoogleCalendarService.php` y `GoogleCalendarGateway.php`.
- Persistencia: modelos `User`, `SocialAccount`, `Cancha`, `Horario`, `Balon`, `Reserva`, `Conversacion`, `MensajeChatbot`, `GoogleCalendarConnection`, `GoogleCalendarEvent` y sus migraciones.
- Seguridad/calidad: Requests, `ReservaPolicy.php`, middlewares, Jobs y pruebas `ChatbotTest.php`, `ChatbotMotorTest.php`, `AnalisisNlpTest.php`, `ReservaTest.php` y `GoogleIntegrationTest.php`.

Para operar la integración se debe mantener el worker con `php artisan queue:work`, revisar `failed_jobs` y `last_error`, limpiar caché de configuración tras cambiar `.env` con `php artisan config:clear`, verificar Calendar desde su panel y ejecutar pruebas antes de desplegar. Como comprobación de este informe se ejecutaron las pruebas focalizadas de Google, chatbot y autenticación: **70 pruebas y 218 aserciones aprobadas**. Nunca se deben copiar claves al documento, repositorio, capturas o frontend.

La explicación puede cerrarse con una frase: **las APIs amplían la experiencia —identidad, comprensión de lenguaje, voz y agenda—, pero Laravel y MySQL conservan el control de permisos, disponibilidad, precios y reservas.** Esa separación permite que el sistema siga siendo coherente y recuperable cuando un servicio externo no está disponible.